

CAD Design Project 2 – Clock Buffer Insertion

Due: 23:59, Oct. 14, 2026

Clock buffer insertion plays a pivotal role in clock tree synthesis (CTS), which distributes a synchronous clock with bounded fanout and wire length budgets while seeking low clock skew and buffer cost. In this project, you are required to design a rectilinear-length-based clock tree by inserting levelized clock buffers hierarchically. Multiple buffer types are available for selection. No actual clock tree routing is required. Your program will be evaluated in a Linux environment (e.g., WSL) according to the following requirements.

- 1 Read a CBI input file with parent-to-child driving constraints, in terms of fanout and total outgoing wire length, and fixed clock pins from the clock source (denoted as SRC) to ordered clock sinks (denoted as S#). Each buffer type has its own fanout limit, length limit, and positive cost. The SRC driving limits are specified separately. For simplicity, the chip dimension is (0, 0) to (dimX, dimY), and all coordinates are integers.
- 2 Generate a CBI output file that lists the ordered clock buffers and the components in each clock tree level. Only SRC and buffers may be parent nodes, and all leaf nodes must be clock sinks. All inserted buffers must be within the chip boundary, and all component coordinates (SRC, sinks, and buffers) must be different. (1) Clock buffers, denoted as B#, are listed with their library types and coordinates. Use consecutive IDs B1 through Bm. (2) SRC is at level 0. Each buffer and sink appears exactly once as a child and is reachable from SRC. The output must form one tree, with no cycles or unused buffers. (3) Component hierarchy is explicitly specified by parent-to-child groupings between adjacent levels (e.g., level-id parent-id1:{child-id1 child-id2...} parent-id2:{child-id1}...). The .level value is the maximum tree level, excluding SRC. Sinks may occur at different levels.

- 2.1 For every parent u, let C(u) be its direct children, and let F_u and L_u be the limits of fanout and total outgoing length. Each child counts as one load regardless of its type. With Manhattan distance $d(u, v) = |x_u - x_v| + |y_u - y_v|$, the following constraints must hold:

$$|C(u)| \leq F_u \quad \sum_{v \in C(u)} d(u, v) \leq L_u$$

- 3 The arrival time of clock sink *i* is the sum of rectilinear (Manhattan) lengths along the parent-to-child edges on the tree from the clock source SRC to the clock sink node *i*, which are contained within the path *P_i*.

$$T_i = \sum_{(u,v) \in P_i} d(u, v)$$

- 4 The clock skew, denoted as T_{skew} , is the difference between the maximum and minimum arrival times among all clock sinks, denoted as T_{max} and T_{min} , respectively.

$$T_{\text{skew}} = T_{\text{max}} - T_{\text{min}}$$

- 5 Your program performance is ranked by Score, the weighted sum of clock skew and total buffer cost. A lower Score ranks better. The total buffer cost is the sum of the library costs of all inserted buffers. Let B be the set of inserted buffers. For each buffer b , c_b is the cost of the buffer. The positive skew penalty weight w_{skew} is specified by `w_skew` in integer.

$$\text{Score} = \sum_{b \in B} c_b + w_{\text{skew}} \times T_{\text{skew}}$$

- 6 Submit a **research report** as a single PDF of at most 3 pages. Explain initial legal solution construction and subsequent Score optimization while preserving legality. Support your strategies with analysis, experimental evidence, and trade-offs. Compare skew, buffer cost, and Score across stages on the same inputs. Figures, tables, a GUI, auxiliary analysis tools, or your own special features are highly encouraged.
- 7 Upload a tarball (*.tgz) of your **C/C++ program source code** and a text **ReadMe file**, which illustrates how your program is to be compiled and executed in a Linux environment.

(NOTE: The uploaded file name should be the same as your student ID.)

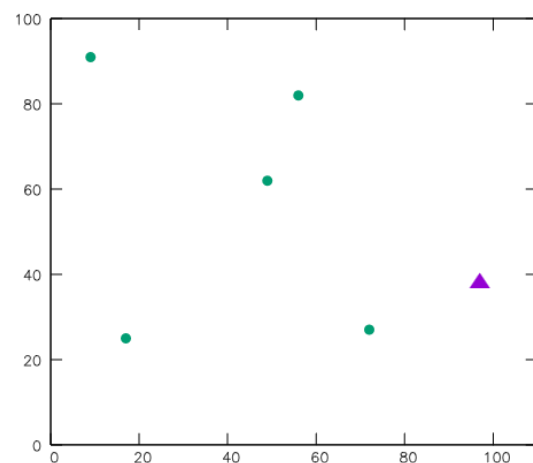
Input File Example: input.cbi

```
.limit
fanout 2      # maximum fanouts for SRC
length 100    # maximum length for SRC
.dimx 110     # chip dimension in x-axis
.dimy 100     # chip dimension in y-axis
.e

.buflib 3
# type  #F  #L  #C #fanout/length/cost
SMALL   3   90  10
MEDIUM 6  120  16
LARGE   9  190  22
.e

.objective
w_skew 1
.e

.pin 6        # number of pins
97 38         # clock SRC coordinates
49 62         # clock sink S1 coordinates
17 25         # clock sink S2 coordinates
56 82         # clock sink S3 coordinates
72 27         # clock sink S4 coordinates
9 91          # clock sink S5 coordinates
.e
```



SYNOPSIS for CBI

```
%> cbi INPUT_FILE OUTPUT_FILE
```

Write the clock tree to OUTPUT_FILE and print T_max, T_min, and Score to standard output.

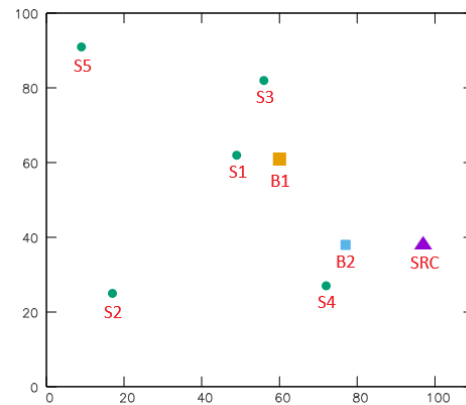
Each .buffer entry gives the buffer ID, type, and integer x and y coordinates. Each line entry k parent:{children} lists edges from level k-1 to level k.

Run-time Examples:

```
%> cbi input.cbi output1.cbi
T_max: 141, T_min: 36, Score: 131
```

```
%> cat output1.cbi
.buffer 2      # number of clock buffers
B1 MEDIUM     60 61   #clock buffer B1
B2 SMALL      77 38   #clock buffer B2
.e

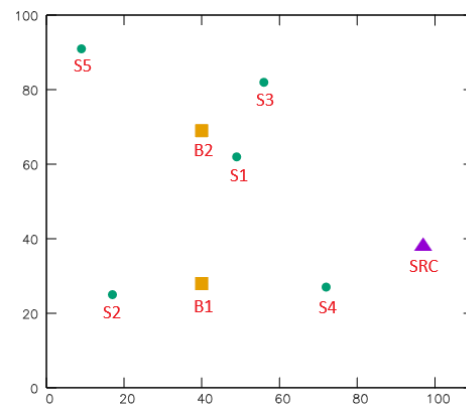
.level 2      # number of tree levels
1 SRC:{B1 B2} # hierarchy
2 B1:{S1 S3 S5} B2:{S2 S4}
.e
```



```
%> cbi input.cbi output2.cbi
T_max: 161, T_min: 93, Score: 100
```

```
%> cat output2.cbi
.buffer 2      # number of clock buffers
B1 MEDIUM     40 28   #clock buffer B1
B2 MEDIUM     40 69   #clock buffer B2
.e

.level 3      # number of tree levels
1 SRC:{B1}    # hierarchy
2 B1:{B2 S2 S4}
3 B2:{S1 S3 S5}
.e
```



```
%> cbi input.cbi output3.cbi
T_max: 141, T_min: 36, Score: 127
```

```
%> cat output3.cbi
.buffer 1      # number of clock buffers
B1 LARGE      49 54   #clock buffer B1
.e

.level 2      # number of tree levels
1 SRC:{S4 B1} # hierarchy
2 B1:{S1 S2 S3 S5}
.e
```

